Práctica 03 – Ejercicio 1: Justificación de la Representación de Relaciones en XML

Asignatura: Gestión de Sistemas de Información (Curso 2026/2027)
Universidad: Universidad Pública de Navarra (UPNA)
Grupo: GSIGR02
Autores: Joel Vílchez, Urko Inés, Aimar Ijurko, Iker Fernández, Pablo Lázaro, Pablo Mosquera, Juan Garbayo

Al pasar las clases del paquete `GSILabs.BModel` a formato XML, es necesario decidir cómo representar las relaciones entre objetos dentro de una estructura de árbol sin provocar ciclos ni duplicar información innecesariamente. Para ello hemos diferenciado dos tipos de relaciones:

1. Relaciones de composición (objetos incluidos)

Los objetos que forman parte directa de otra entidad y no tienen sentido por sí solos en el sistema se incluyen directamente como sub-elementos dentro del XML. Este es el caso de `Direccion` dentro de `Local` (y en sus subclases `Bar`, `Pub` y `Restaurante`), así como la lista de especialidades en `Bar` o las horas de apertura y clausura en `Pub`. De esta forma, al guardar un local se conserva toda su información básica en un único bloque.

2. Relaciones entre entidades independientes (`Review`, `Reserva` y `Contestacion`)

Las clases `Review`, `Reserva` y `Contestacion` relacionan usuarios (`Cliente`, `Dueño`) con locales (`Local`, `Reservable`). Para representar estas asociaciones hemos tomado dos decisiones:

- **Los locales y clientes no incluyen sus reseñas ni reservas:** En el diseño de `BModel`, ni `Local` ni `Cliente` guardan listas de `Review` o `Reserva` (esa relación se gestiona en `BusinessSystem`). Si un local incluyera todas sus reseñas y cada reseña incluyera a su vez el local, se produciría un bucle infinito y se repetirían los datos continuamente.
- **Las clases asociativas sí incluyen los objetos que relacionan:** Tanto `Review` como `Reserva` y `Contestacion` contienen en sus atributos las referencias a los objetos que conectan (`<cliente>`, `<local>`, `<dueño>` o `<review>`), por lo que los anidan dentro de su propio XML. Así conseguimos que cualquier reseña, reserva o contestación guardada por separado con `saveToXML()` tenga toda la información necesaria para poder leerse y reconstruirse de forma independiente sin depender de otros ficheros.

3. Conclusión

Con este criterio se respeta la estructura actual de las clases de `BModel`, se evitan referencias circulares y cada objeto puede guardarse y recuperarse en XML de manera independiente.